

Windows Registry Persistence Detection and Remediation Lab

Hands On Cybersecurity Portfolio Project

Field	Details
Author	Bemvindo Ntambu
Environment	Windows 10 virtual machine
Primary tools	PowerShell and Microsoft Sysinternals Autoruns
MITRE ATT&CK	T1547.001 Registry Run Keys and Startup Folder
Completion status	Completed and remediated successfully
Final validation	PASS

Document Purpose

This report documents a complete, authorized Windows persistence investigation performed in an isolated virtual machine. The project used a harmless Registry Run value that launched Microsoft Notepad at user logon. The entry was baselined, detected, validated, removed, and verified using PowerShell, Registry queries, Sysinternals Autoruns, and controlled logon tests.

The final result was successful: the persistence mechanism executed as expected during the test, was removed without affecting unrelated startup entries, and did not execute after remediation.

Project Summary

Windows Registry Run keys are legitimate startup mechanisms that can also be abused for persistence. This lab demonstrates how an analyst can identify an unexpected startup value, validate its behavior, preserve evidence, remove it, and confirm that the endpoint no longer exhibits the behavior.

Objective	Outcome
Create a controlled persistence simulation	Notepad configured under the current-user Run key
Detect the startup entry	Located with reg.exe, PowerShell, and Autoruns
Validate the behavior	Notepad opened automatically after logon
Remove the persistence mechanism	Registry value removed successfully
Verify remediation	Autoruns result empty and Notepad did not relaunch
Preserve evidence	Screenshots and text-based results retained

Skills Demonstrated

- Windows Registry analysis and startup-location review
- PowerShell evidence collection and remediation
- Microsoft Sysinternals Autoruns and Autorunsc
- Behavioral validation through controlled logon testing
- Evidence preservation and privacy-aware documentation
- Incident-response workflow and post-remediation verification
- MITRE ATT&CK mapping and portfolio reporting

Environment and Tools

Component	Use in the lab
Windows 10 VM	Isolated endpoint used for safe testing
VirtualBox snapshot	Recovery point created before changes
PowerShell	Created, queried, documented, and removed the Registry value
Windows Registry	Current-user Run key used for the simulation
Autoruns	GUI-based detection and remediation verification
Autorunsc	Command-line baseline collection
Notepad	Harmless executable used to demonstrate logon execution

Complete Lab Procedure

Step 1 Create the Recovery Point

A VirtualBox snapshot named Before Registry Forensics Lab was created before modifying the VM. This provided a known recovery point and limited the risk of unintended configuration changes.

Step 2 Create the Lab Directories

The following working structure was created:

```
C:\Registry-Forensics-Lab\  
├── Evidence  
├── Exports  
├── Reports  
└── Scripts
```

Step 3 Record the Initial Baseline

System information, the current Registry Run-key contents, and an Autoruns baseline were collected before introducing the test value. The full raw baseline was retained privately because it can contain usernames, installed applications, and system-specific paths.

```
reg.exe query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
```

Step 4 Install and Verify Autoruns

Microsoft Sysinternals Autoruns was downloaded to C:\Tools\Autoruns. The Authenticode signature on Autoruns64.exe was validated before execution.

```
Get-AuthenticodeSignature "C:\Tools\Autoruns\Autoruns64.exe" |  
Select-Object Status, StatusMessage, @{Name="Signer";Expression={$_.SignerCertificate.Subject}}
```

Observed result: Valid signature issued to Microsoft Corporation.

Step 5 Capture the Autoruns Baseline

Autorunsc was used to capture the endpoint startup configuration before persistence was introduced. Raw and cleaned CSV copies were retained for private analysis but excluded from the public portfolio package.

Step 6 Create the Harmless Persistence Entry

A current-user Registry Run value was created to launch Notepad during login. The simulation did not use malware, scripts, downloaders, or external payloads.

```
$RunKey = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
```

```
New-ItemProperty `
-Path $RunKey `
-Name "RegistryLabPersistence" `
-Value "C:\Windows\System32\notepad.exe" `
-PropertyType String `
-Force
```

Step 7 Confirm the Registry Value

```
reg.exe query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" `
/v RegistryLabPersistence
```

Observed value:

```
RegistryLabPersistence    REG_SZ    C:\Windows\System32\notepad.exe
```

Step 8 Detect the Persistence Entry with Autoruns

Autoruns was opened as Administrator. The Logon category was selected, the filters that hide Windows and Microsoft entries were cleared, and RegistryLabPersistence was entered in the search field. Autoruns displayed the value under the current-user Run key and verified that the target executable was signed by Microsoft.

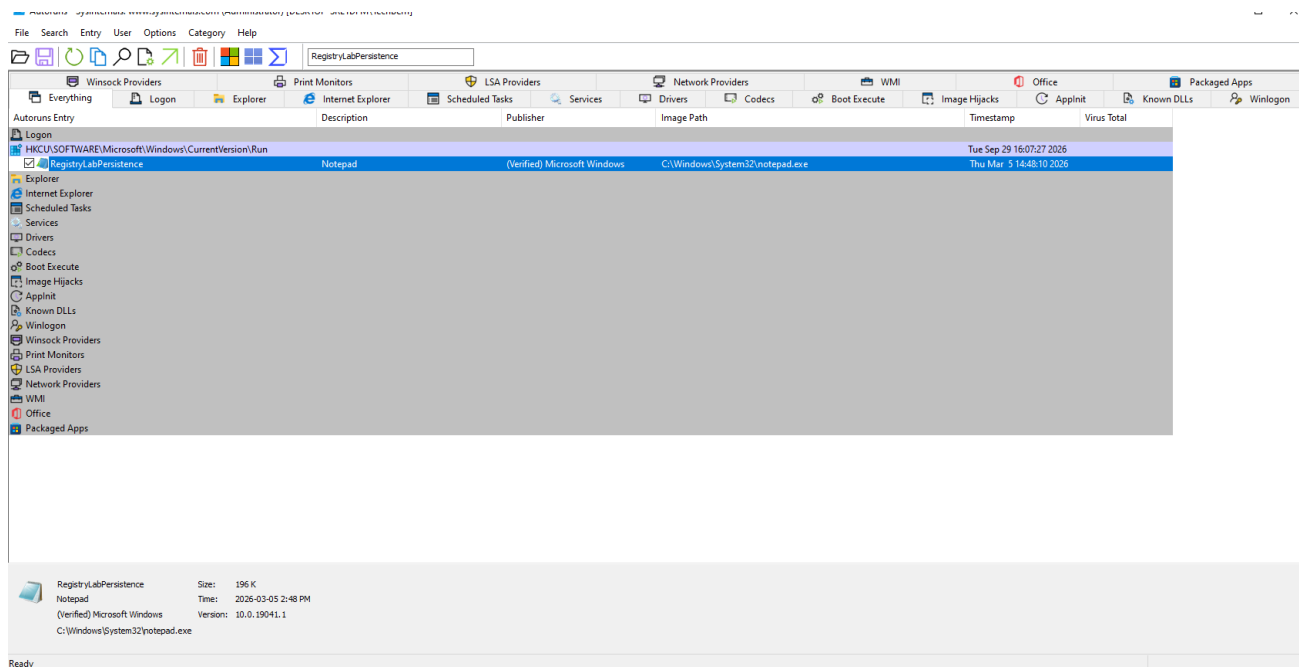


Figure 1 Detection evidence Autoruns displays RegistryLabPersistence under the current user Run key

Step 9 Validate Execution at Logon

The user signed out and signed back into the same Windows account. Notepad opened automatically within the expected logon period, confirming that the Registry value provided functional persistence.

Test condition	Observed result	Status
Registry value present	RegistryLabPersistence returned by query	PASS
User signs in	Notepad opened automatically	PASS
Autoruns inspection	Entry visible under Logon	PASS

Step 10 Preserve Detection Evidence

The following evidence was recorded before remediation:

- Autoruns detection screenshot
- Registry query output
- Behavioral logon-test result
- Timestamped persistence-test result

Step 11 Remove the Persistence Entry

```
Remove-ItemProperty `
-Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" `
-Name "RegistryLabPersistence" `
-ErrorAction Stop
```

The removal completed successfully without modifying unrelated startup values.

Step 12 Verify the Registry Remediation

A follow-up Registry inspection confirmed that RegistryLabPersistence no longer existed. The absence of the value was treated as the expected verification result rather than an investigation error.

Step 13 Verify the Autoruns Remediation

Autoruns was refreshed while the RegistryLabPersistence filter remained active. The results pane was empty, providing visual confirmation that the logon entry had been removed.

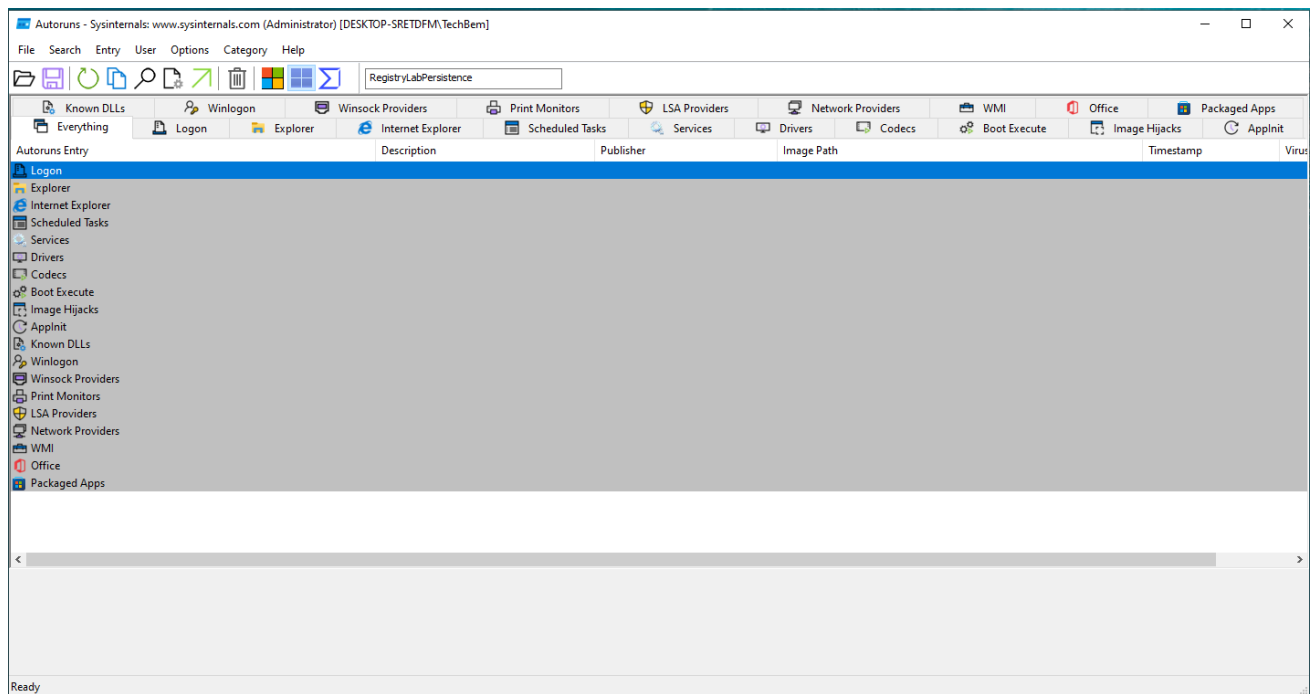


Figure 2 Remediation evidence Autoruns returns no result for RegistryLabPersistence after removal

Step 14 Perform the Final Logon Test

The user signed out and signed back in after remediation. Notepad did not open automatically. This confirmed that the persistence behavior was no longer active.

Verification method	Expected result	Observed result	Status
Registry inspection	Value absent	Value absent	PASS
Autoruns filter	No matching entry	No matching entry	PASS
Post-remediation logon	Notepad does not open	Notepad did not open	PASS

Investigation Findings

Field	Finding
Registry path	HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Value name	RegistryLabPersistence
Value type	REG_SZ
Target executable	C:\Windows\System32\notepad.exe
Execution trigger	User logon
Executable signature	Verified Microsoft Windows

Field	Finding
Initial behavior	Notepad opened automatically
Remediated behavior	Notepad did not open automatically
Final result	PASS

MITRE ATTACK Mapping

Category	Mapping
Tactic	Persistence
Technique	Boot or Logon Autostart Execution
Sub technique	T1547.001 Registry Run Keys and Startup Folder

Reference: [MITRE ATTACK T1547.001](#)

Tool reference: [Microsoft Sysinternals Autoruns](#)

Security Recommendations

- Monitor both current-user and local-machine Run and RunOnce keys.
- Alert on newly created startup values that reference unusual paths, scripts, temporary folders, or user-writable locations.
- Validate digital signatures, file hashes, parent processes, command lines, and creation timestamps.
- Compare current startup configurations against a trusted baseline.
- Preserve relevant evidence before removing a suspicious persistence mechanism.
- Use least privilege and application-control policies where operationally appropriate.
- Verify remediation through both configuration inspection and behavioral testing.

Lessons Learned

Registry persistence analysis requires more than locating a value. A defensible investigation connects the configuration to its target executable, verifies whether the behavior occurs at logon, preserves evidence, removes only the identified entry, and then uses independent methods to confirm that the behavior has stopped.

Autoruns filtering options can hide legitimate Microsoft entries. During authorized testing, the analyst must understand those filters and refresh the tool after changes. Public evidence must also be reviewed for usernames, hostnames, software inventory, email addresses, IP addresses, and other environment-specific identifiers.

Evidence Placement Guide

The two screenshots are already inserted in this Word document. If they are ever replaced, use the following locations so the report remains logically ordered.

Picture	Filename	Location in this report	Purpose
Picture 1	01-autoruns-persistence-detected.png	Immediately after Step 8	Shows the active persistence entry
Picture 2	02-persistence-removed.png	Immediately after Step 13	Shows the empty result after remediation

The raw Autorunsc or Notepad-output image is optional and should not be added to the public report because it can reveal usernames, software inventory, and user-profile paths.

Convert the Word Report to PDF

1. Open the completed Word document and verify both images, captions, page breaks, and command blocks.
2. Select File, then Save As or Export.
3. Choose PDF as the file format.
4. Select Standard publishing quality.
5. Save the PDF as Windows_Registry_Persistence_Detection_and_Remediation_Lab.pdf.
6. Open the PDF and verify every page before publishing it.

Recommended Public Repository Structure

```
windows-registry-persistence-lab/  
├── README.md  
├── report/  
│   └── Windows_Registry_Persistence_Detection_and_Remediation_Lab.pdf  
├── evidence/  
│   ├── 01-autoruns-persistence-detected.png  
│   └── 02-persistence-removed.png
```

Create and Publish the GitHub Repository

Part 1 Prepare the Upload Files

7. Create a folder on the host computer named windows-registry-persistence-lab.
8. Inside it, create folders named report and evidence.
9. Place the final PDF in the report folder.
10. Place the two sanitized PNG screenshots in the evidence folder.
11. Do not upload raw Autoruns CSV files, system-information files, passwords, tokens, email addresses, or unrelated software inventory.

Part 2 Create the Repository

12. Open <https://github.com> and sign in to the ntambu94-ai account.
13. Select the plus icon in the upper-right corner and choose New repository.
14. Set the repository name to windows-registry-persistence-lab.
15. Enter the description shown below.
16. Choose Public so recruiters and portfolio visitors can view the project.
17. Select Add a README file.
18. Do not add a license unless you specifically want to license the report and images for reuse.
19. Select Create repository.

Detection, investigation, and remediation of Windows Registry Run-key persistence using PowerShell and Microsoft Sysinternals Autoruns.

Part 3 Edit the GitHub README

Open README.md, select the pencil icon, and add this concise project summary:

```
# Windows Registry Persistence Detection and Remediation Lab

This hands-on lab demonstrates the detection, behavioral validation, removal, and post-remediation
verification of a Windows Registry Run-key persistence mechanism. The investigation used PowerShell,
reg.exe, Microsoft Sysinternals Autoruns, and controlled logon tests in an isolated Windows 10
virtual machine.

## Results

- Persistence successfully created with a harmless Notepad executable
- Registry value detected with multiple tools
- Logon execution confirmed
- Persistence entry removed successfully
- Autoruns and final logon testing confirmed remediation
- Final result: PASS

## MITRE ATTACK

T1547.001 Registry Run Keys and Startup Folder

## Full Report

[Download the complete lab
report](report/Windows_Registry_Persistence_Detection_and_Remediation_Lab.pdf)

## Evidence

### Detection
![Autoruns persistence detection](evidence/01-autoruns-persistence-detected.png)

### Remediation
![Autoruns remediation verification](evidence/02-persistence-removed.png)
```

Part 4 Upload the Report

20. Open the repository and select Add file, then Upload files.
21. Drag the report folder containing the final PDF onto the upload page.
22. Confirm that the destination path is
report/Windows_Registry_Persistence_Detection_and_Remediation_Lab.pdf.
23. Enter the commit message Add completed registry persistence lab report.
24. Select Commit changes.

Part 5 Upload the Evidence Pictures

25. Return to the repository and select Add file, then Upload files.
26. Drag the evidence folder containing both PNG files onto the upload page.
27. Confirm that the filenames remain 01-autoruns-persistence-detected.png and 02-persistence-removed.png.
28. Enter the commit message Add detection and remediation evidence.
29. Select Commit changes.

Part 6 Verify the Published Project

30. Return to the repository homepage.
31. Confirm that the README displays the project summary.
32. Open the PDF and verify that it downloads or displays correctly.
33. Confirm that both evidence images render inside the README.
34. Check the repository for personal information, credentials, tokens, email addresses, and unrelated system data.

Part 7 Add Repository Topics

In the About section, select the gear icon and add these topics:

```
windows-forensics
registry-forensics
persistence
powershell
sysinternals
autoruns
incident-response
mitre-attack
cybersecurity-lab
```

Part 8 Pin the Project to the Profile

35. Open <https://github.com/ntambu94-ai>.
36. Select Customize your pins.
37. Choose windows-registry-persistence-lab.
38. Save the selection.

Final Publication Checklist

- The PDF opens correctly and contains both evidence figures.
- The README summary accurately describes the completed work.
- The detection and remediation images render correctly.
- No raw baseline CSV or system-information file is public.
- No password, credential, token, email address, IP address, or unnecessary personal identifier is visible.
- The MITRE ATT&CK mapping is included.
- The repository is public and pinned to the GitHub profile.

Conclusion

This project demonstrates practical endpoint investigation and incident-response skills across the full lifecycle of a Windows persistence event. The test established a measurable behavior, used multiple methods to detect and validate it, preserved relevant evidence, applied a targeted remediation, and independently confirmed that the behavior stopped. The completed report and evidence are suitable for inclusion in a cybersecurity portfolio after the final privacy review.